

TASK 1: PYTHON AND OOPS

1. Least Required Coins

Write a Python program that accepts an amount in coins and calculates the minimum number of coins needed to make up that amount using the following denominations:

coins = [25, 10, 5, 2, 1]

Enter the amount: 87

Output: THE MINIMUM COINS NEEDED ARE: 6

2. Recursive Number Operations

Write a Python program with recursive functions to perform the following operations on a given integer:

- factorial(n)
- sum_of_digits(n)
- reverse_number(n)
- fibonacci(n)

Handle invalid inputs and negative values appropriately.

3. Student Record Management with Error Handling

Write a Python program to manage student records using a dictionary.

Each student record should contain:

- Roll number
- Name
- Marks

Implement functions to:

- a) add_student()
- b) search_student()
- c) update_marks()
- d) display_student()

Handle the following errors using appropriate exception handling:

- Duplicate roll number
- Student not found
- Invalid marks
- Invalid input type

Provide a menu-driven interface for the operations.

4. Stack Implementation

Write a Python program to implement a Stack using a class.

Class Stack should contain the following methods:

- push(x)
- pop()
- peek()
- display()

Implement the stack using a list, but do not use the built-in append() or pop() methods.

Raise appropriate exceptions for:

- Stack overflow
- Stack underflow
- Invalid input

Implement a menu-driven program with options.

5. Linked list Implementation

Write a Python program to implement a Singly Linked List using classes.

Create:

- Node class
- LinkedList class

The LinkedList class should contain the following methods:

- insert_beginning(x)
- insert_end(x)
- delete_beginning()
- delete_end()
- search(x)
- display()

Handle deletion from an empty list and searching for unavailable elements using appropriate error handling.

Implement a menu-driven program with options.

6. Create a Python program for a Library Management System.

a) Create an abstract class LibraryMember with methods issueBook(), returnBook() and displayDetails().

b) Create a class Book with attributes bookId, title, author, price and available.

c) Create StudentMember and FacultyMember derived from LibraryMember with suitable additional attributes.

- d) Implement the required methods using method overriding.
- e) Use encapsulation for book availability and raise exceptions for invalid book or member IDs and unavailable books.
- f) Store books and members as objects and implement a menu-driven program with options.

7. Create a Python program for a Vehicle Rental System.

- a) Create an abstract class Vehicle with attributes vehicleNumber, brand, rentPerDay and abstract method calculateRent().
- b) Create Car, Bike and LuxuryCar derived from Vehicle.
- c) Override calculateRent(days) for each vehicle type.
- d) Create a Customer class containing customer details and the rented vehicle.
- e) Raise exceptions for invalid rental duration and unavailable vehicles.
- f) Implement a menu-driven program for adding vehicles, renting vehicles, returning vehicles and displaying vehicle details.

8. Create a Python program for an Online Shopping Cart.

- a) Create a class Product with attributes productId, name, price and stock.
- b) Create an abstract class Payment with method makePayment() and derive CardPayment and UPIPayment.
- c) Create a ShoppingCart class that maintains products and their quantities.
- d) Implement adding products, removing products and calculating the cart total.
- e) Raise exceptions for invalid products, insufficient stock and failed payments.
- f) Demonstrate abstraction, composition, method overriding and polymorphism.

9. Hospital Appointment System

Create a Python program for a Hospital Appointment System.

- a) Create a base class Person with attributes id, name and age.
- b) Derive Doctor and Patient from Person with suitable additional attributes.
- c) Create an Appointment class containing appointment ID, doctor, patient, date and time.
- d) Implement booking, cancellation and display operations.
- e) Prevent multiple appointments for the same doctor at the same date and time.
- f) Raise exceptions for invalid IDs and unavailable appointment slots.
- g) Demonstrate inheritance, composition, encapsulation and exception handling.